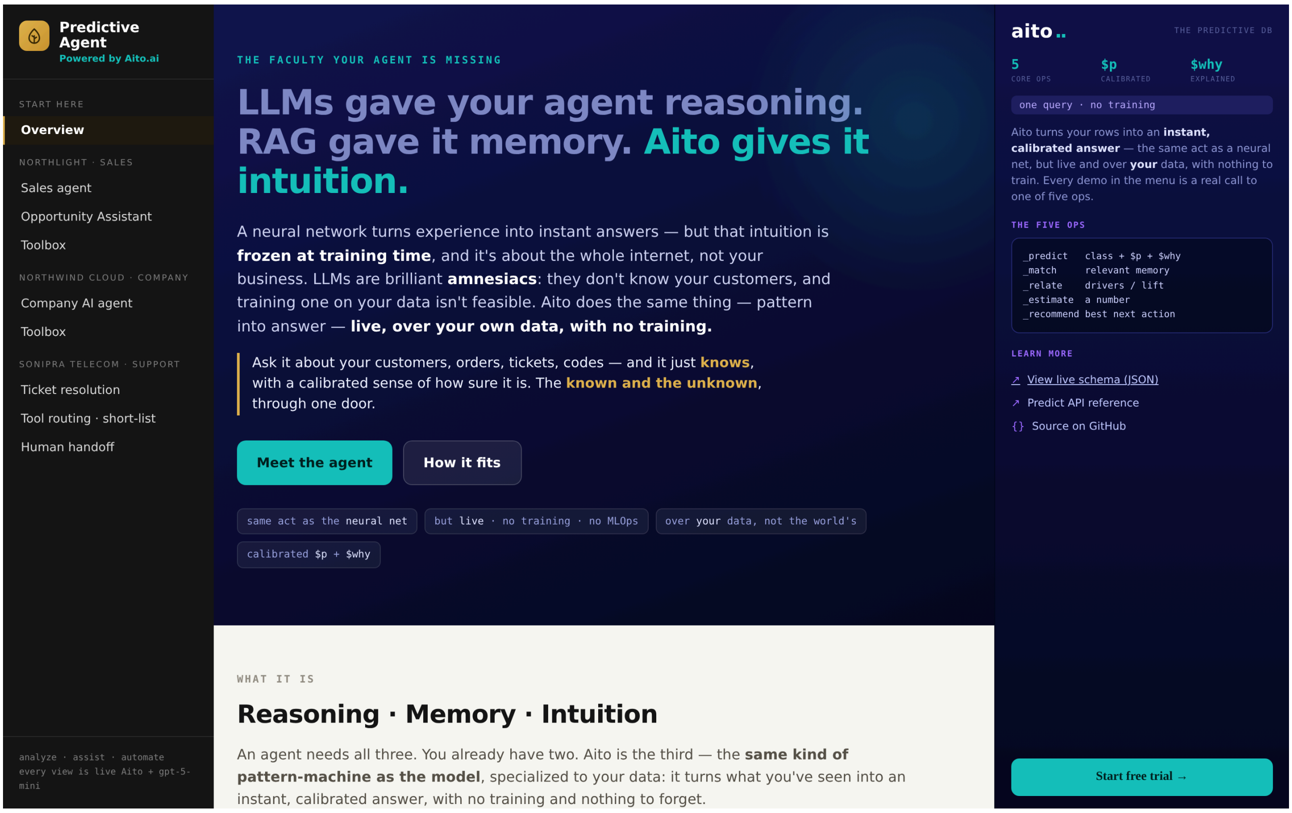
The faculty your agent is missing

LLMs gave your agent reasoning. RAG gave it memory.
Aito gives it intuition.

A live agent that calls Aito ops as tools — grounded numbers it can't invent, and the action that wins. Three working surfaces (sales, company 360, support), one predictive database, no model training. Better, faster, cheaper — and higher-yield.



Where a capable agent quietly falls down

None of these mean you picked the wrong model or built the wrong platform. They're the predictable failure modes of asking one LLM to reason *and* remember *and* do arithmetic over a large, structured, ever-changing dataset. Each has a one-query fix.

🧰 **Tool / option sprawl**
Hundreds of tools or SKUs in context → selection degrades, prompts bloat.

🐌 **An LLM call on every step**
Multi-step workflows take seconds and burn tokens, per ticket, at scale.

🎯 **Vector search misfires**
Embeddings dilute identifiers — the nearest neighbour is the wrong customer.

🔢 **Bad with numbers**
Aggregation, drivers, estimates — the model guesses, often confidently wrong.

⚖️ **No sense of "how sure"**
Overconfident output gives no signal for when to act vs ask a human.

🧠 **Amnesiac about your data**
It knows the internet, not your customers — and you can't feasibly train it on them.

The solution: Aito is a tool in the agent's toolbox

Aito is a predictive database. Load your history; query for predictions, recommendations and statistics through SQL-like calls. **No model training, no retraining schedule, no MLOps.** Every answer carries a calibrated probability (`$p`) and an explanation (`$why`) that traces straight to your data.

The agent keeps doing the reasoning and the talking. When it needs a number it can't invent — win odds, effort, churn, the action that wins — it **calls an Aito op as a function.** The same agent with the Aito tools switched off has to guess, and says so. That is the whole pitch, A/B in one toggle.

- `_predict` — a class + calibrated \$p + \$why
- `_match` — the relevant memory, by structure
- `_relate` — drivers / lift behind a number
- `_estimate` — a number, from history
- `_recommend` — the next action that maximises a goal

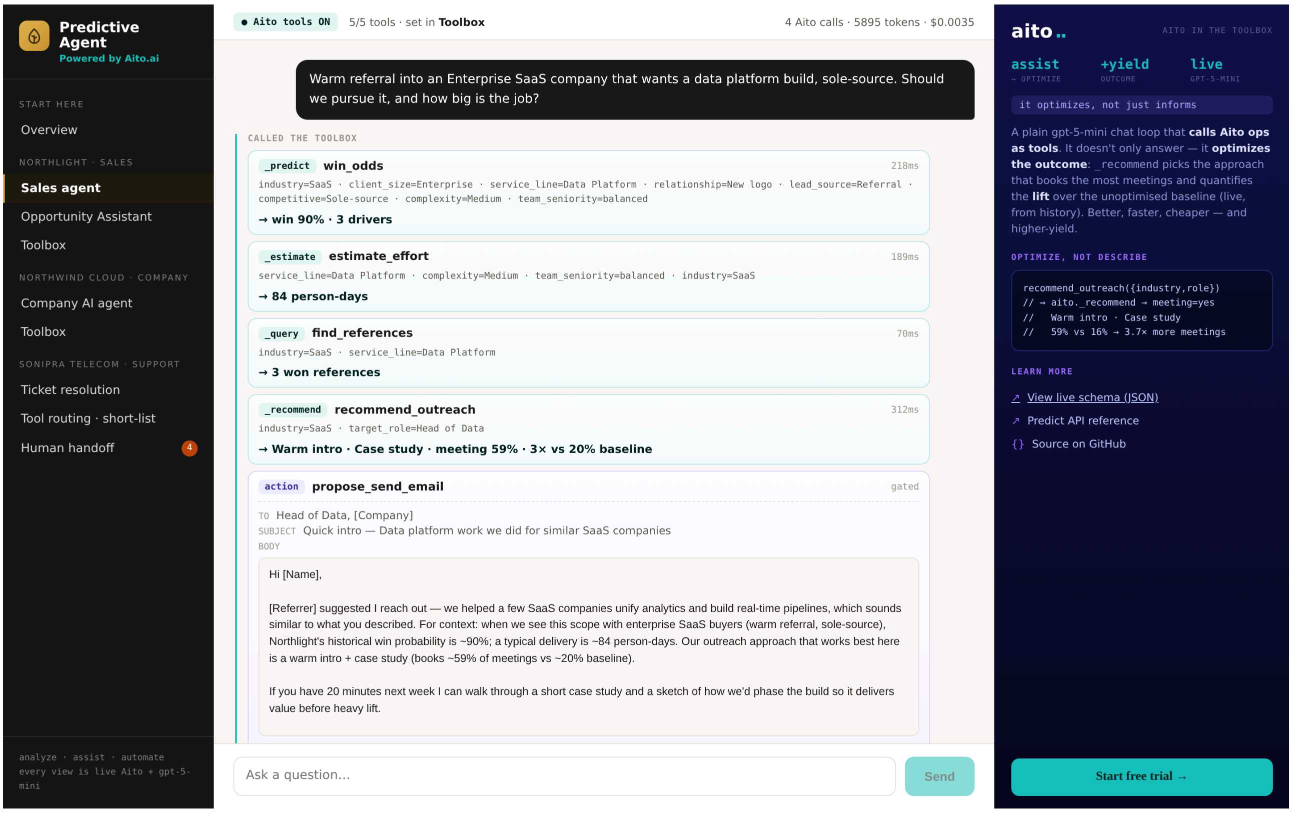
One query, any language. Call it like a tool or an MCP endpoint from any agent framework. Your platform stays the brain; Aito is the instant, calibrated memory underneath it.

Zero MLOps. No model files, no retrain, no drift. A row added today is in the next prediction — which is also how the agent *learns*: log an outcome and the very next recommendation is sharper.

Calibrated & explainable. Every answer has a `$p` and a `$why`. Auditable by design — the `$p` is a governance gate: auto when sure, escalate when not.

Aito in the toolbox — the sales agent

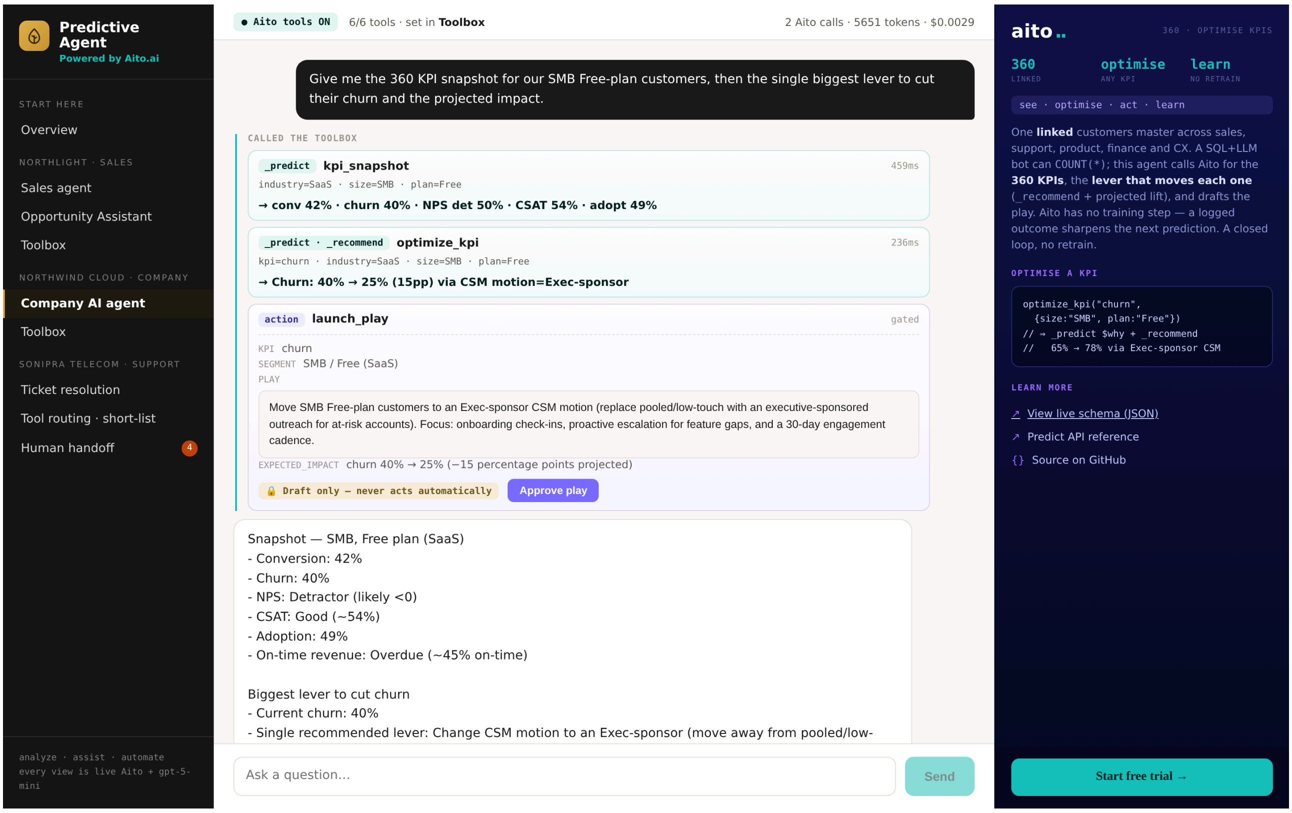
A live gpt-5-mini chat loop. Ask it about a deal; it reasons, then calls Aito ops for the numbers it can't invent: win-odds (`_predict` + `$why`), effort (`_estimate`), reference projects (`_query`), and the outreach that books the most meetings (`_recommend`) — quoting the **lift over the unoptimised baseline**. Every tool call is rendered inline, so you watch Aito work inside the agent. Money/state actions are gated: it drafts the email, it never sends.



Toolbox on/off · grounded numbers vs flagged guesses · 4 Aito calls in 0.8 s, $0 · the LLM call reasons; Aito supplies the facts.

A 360° copilot that optimises KPIs — the company agent

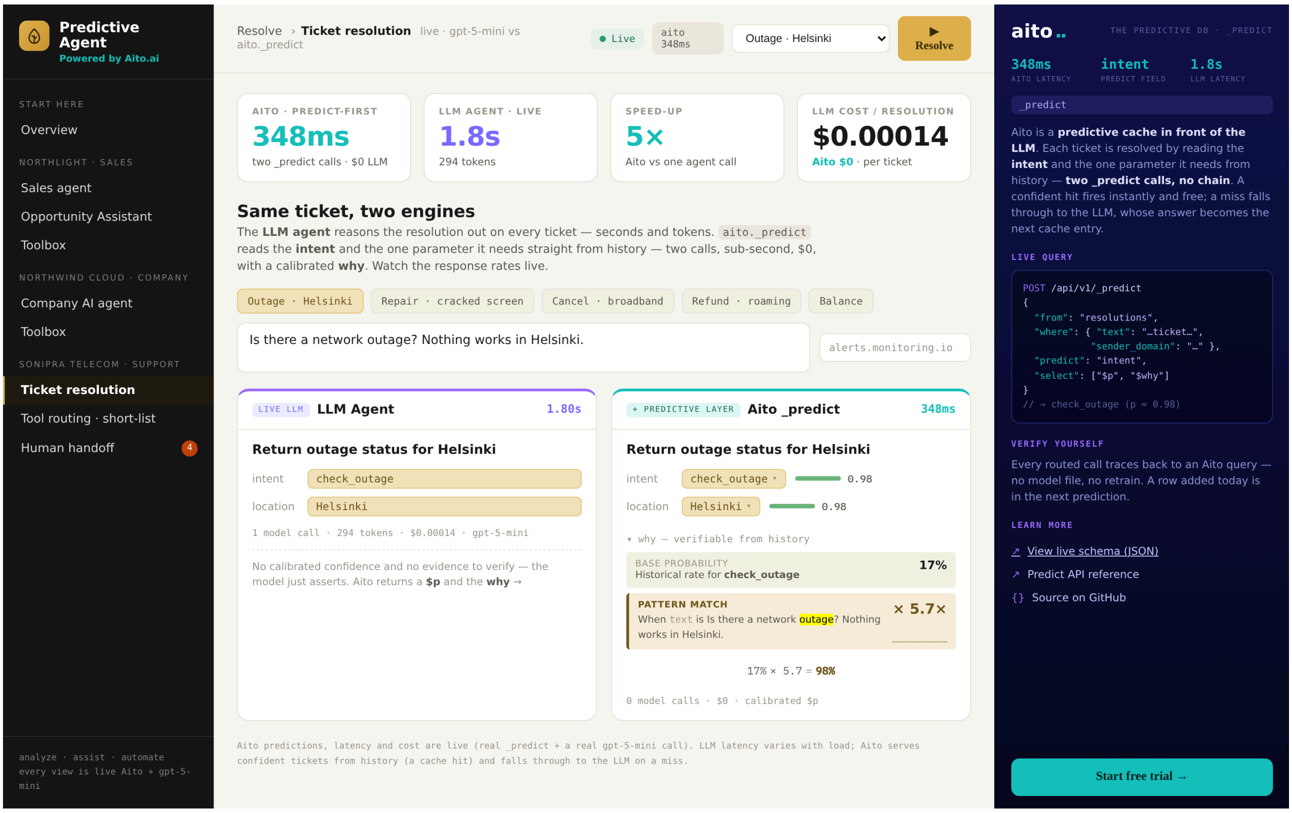
One linked `customers` master joined to sales, support, product, finance and CX. A SQL+LLM BI bot can `COUNT(*)`; this copilot calls Aito for the **360 KPI snapshot** (conversion, churn, NPS, CSAT, adoption, on-time revenue), the **single lever that moves each one** with the projected lift, and a **customer-360 join** across every domain. It doesn't just report — it optimises, drafts the play (gated), and learns: Aito has no training step, so a logged outcome sharpens the next prediction.



“Churn 40% ⟶ 25% via an Exec-sponsor CSM motion” — driver, lever and projected lift, computed live from the company’s own linked data. See · optimise · act · learn.

Predict-first support — the resolution console

The same ticket, two engines, side by side. An LLM agent reasons the resolution out on every ticket — seconds and tokens. `aito._predict` reads the intent and the one parameter it needs straight from history — two calls, sub-second, $0, with a calibrated `$why`. A confident hit is served instantly (a cache hit); a miss falls through to the LLM, whose answer becomes the next cache entry. The `$p` gate routes the unsure ones to a human, and anything sensitive (refund, cancel) is gated regardless.



Benchmarked, not asserted

Three failure modes every agent team hits — each run as a real benchmark (live Aito
1. live gpt-5-mini on seeded, realistic data) against the tool a good engineer would

otherwise reach for.

01 Shortlisting holds at scale

Embedding-retrieval shortlist degrades as the catalog grows; Aito holds and hands the LLM ~16× fewer tokens for the same pick.

02 ~9–10× lower latency

A 6-step LLM resolution chains to ≈22 s; Aito predicts in parallel in ~0.15 s — resolved before the agent clears step one.

03 Right context-memory

Vector search picks the wrong customer's memory 86% of the time; Aito conditions on structure and recovers what the text can't identify.

See it just know — all three agents, live.

Real predictions, real latency, real cost — at **agent.aito.ai**. A primitive your agents call, not another platform to adopt.